Metastore
↓
Catalog
↓
Schema (Database)

- Data: Table
  - Managed
  - External
- View
- Volume (non tabular)
  - Managed
  - External
- Function
- Model (ML)

Why UC?

1. Unified discovery and governance for all data and AI assets, any format, any data source
2. Open to any tool or engine for a broad ecosystem interoperability and sharing
3. Powers data intelligence using semantic, lineage and usage for platform optimizations.

Privilege

Table

Principal	Privilege
analysts	SELECT
dbadmins	ALL PRIVILEGES

→ allows domain owners to explicitly to manage their assets

Centralized access control

Using ANSI SQL DCL (Data Control Language)

→ American National Standards Institute Structured Query Language

GRANT <privilege> ON <securable_type> <securable_name> TO '<principal>'

GRANT SELECT ON iot.events TO engineers

Choose permission level

'Table' = collection of files in S3/Azure

sync groups from your identity provider

Using UI

On "Catalog Explorer"
Select under "Users and Groups"
Select under "Privileges" incl. SELECT, MODIFY, ALL PRIVILEGES

Programmatically

Databricks CLI (Command Line Interface), Terraform and REST APIs

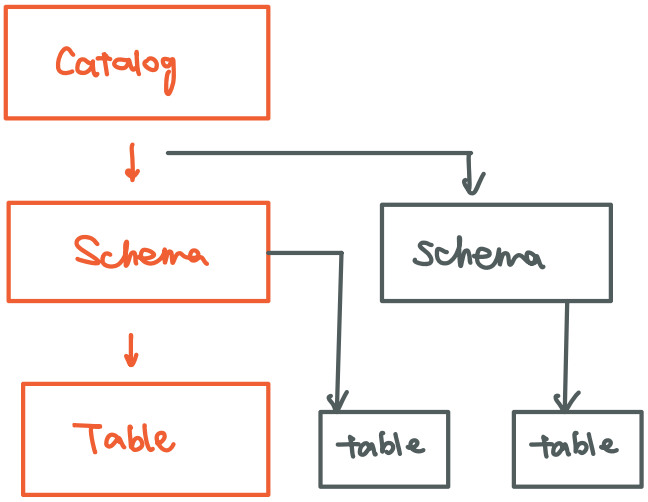
Privileges ↔ Securables ↔ Principals

Securable = catalog, schema, table, view, function, share, volume, model, etc

✓ USE CATALOG

✓ USE SCHEMA

✓ SELECT



The access can be inheritted from schema or catalog

Roles

Foundation of architecture

Account Admin

administrates the entire Databricks account primarily through the account console

- **Workspace Admin**
  - IT / Platform / DevOps team
  - Regular Service Principal
  - Audits

  Manage workspace access controls, settings, and features for individual workspace in the account

  ↓ **Account user**

- **Metastore Admin**
  - Platform Ops or Central Gov Team
  - Delegate ownership to data owners

  can manage all objects in the metastore

  Bind catalogs to workspaces

  ↓ **Catalog Owner**

  can manage a catalog and its schema and schema objects,

  Bind catalogs to workspaces

  - BU Gov Team or central Gov Team

  ↓ **Schema Owner**

  can manage a schema and its objects

  ↓ **Table / View owner**

- **Marketplace Admin**
  - Platform Ops or Central Gov Team

  Manage Marketplace provider profile, incl. creating and managing Marketplace listings

  ↓ **Marketplace Users**

4 Policy Types
- Now available: Row Filter / Column Mask
- Coming soon: GRANT / DENY

Owners or users with MANAGE permission can apply policies to UC objects

Data Catalog Hierachy

Metastore

Catalog
- top-level in catalog tab
- root container in hierarchy
- holds multiple schemas

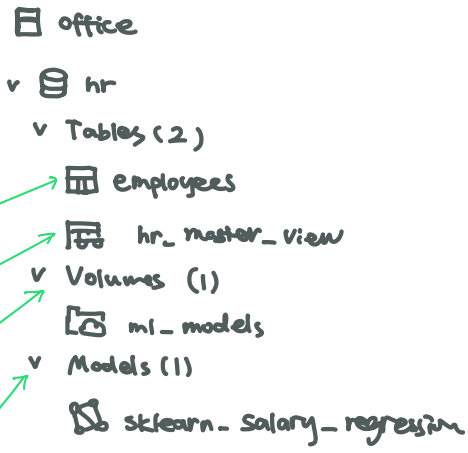
Schema
- nested under a catalog
- groups related assets
- Organizes tables, views, models, functions

Volumes
- Separate
- Used for unstructured data

Models/ Function
- Reusable ML and the analytical assets

Views
- Represent query outputs

Managed/ Ext tables
- Listed under Tables in schema
- Differentiation in details pane
- Core data storage objects



Lineage is accessible by navigating to the Catalog → Data Object → Lineage

Feature table

- Located in AI/ML → Features
- Feature tables searchable/ listed with metadata and workflows
- Same feature table also accessible under schema in Catalog Explorer

Attribute-Based Access Control + Tags

- Central policies across all data + AI assets w/UI + SQL interface
- Control access using governed tags applied at the catalog, schema, or table level
- Tag-based policy types
  - fine-grained policies (RLS[Role Level Security] / CLM [Contract Lifecycle Management]) on any catalog/ schema/table/view
  - Coarse-grained (GRANT/DENY) on any catalog/schema/table/ view

```
SET RULE col_mask
ON CATALOG turbine_prod
COLUMN MASK masking_function
TO 'account users'
FOR tables
WHEN COLUMN tagged ('pii')
```

Governed Tags Use Case

Access Management

- Governed Tags determine which tags can be referenced in ABAC
- Adds governance over existing tags used to restrict access

Cost Attribution

- Integrating with Budget Policies
- Ensures consistent and accurate usage of tags for cost monitoring

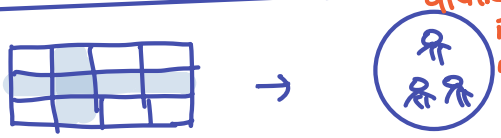
Dorfabricks System Tags

- Certification and Deprecation
- Auto Classification
- Workspace level tags

Fine-grained Access Control

Row Column Security

Applies policies directly at the source table, offering better performance, maintainability, and scalability



Native row and column security attaches policies directly to the base table metadata, providing cen-tralized enforcement regardless of the query path, making it the most scalable and managable solution for enterprise governance

```
CREATE FUNCTION function_name (param, param2, ...)
RETURN {filter clause which returns a boolean};

CREATE FUNCTION us_filter( geo STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('admin'),
        true, geo = 'US')
```

test for group membership

Define filter predicates

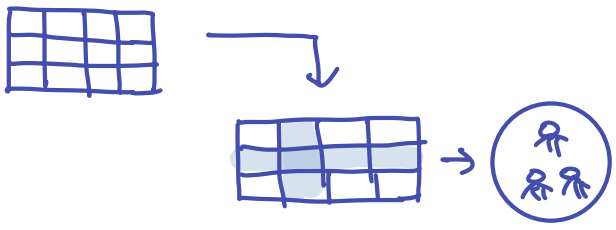
```
ALTER TABLE sales
SET ROW FILTER us_filter
ON (region);
```

← Assign reusable filter to table

- 'admin' group can access all rows in region column
- other groups only see rows when region = 'US'

Dynamic View

Complexity



```
CREATE FUNCTION function_name (param, param2, ...)
RETURN {output matches type of first parameter};

CREATE FUNCTION ssn_mask(ssn STRING)
RETURN CASE WHEN
        is_account_group_member('HRDept')
        THEN ssn
        ELSE '***-**-****'
END;
```

Test for group membership

Define logic for masking

```
ALTER TABLE users
ALTER values for ssn
SET MASK ssn_mask;
```

← Assign reusable mask to column

- Members of 'HRDept' group can access all column values for ssn
- Other groups see '--***' for the ssn colum

Dynamic Views compared to Row and Column Security

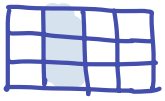
Downsides

- Need to manage multiple objects
- More difficult to maintain
- Views are read-only

When to use them?

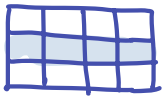
- Read-only tables
- Need to abstract source table
- Need to present information from more than one table

Limit Access to Column



Omit column values from output

Limit Access to Row



Omit rows from output

Data Masking

obscure data

..... @databricks.com

Can be conditional on a specific user/service principal or group membership through Databricks-provided functions